手写小GPT学习笔记

一、Multi-Head Attention 的核心作用

Multi-Head Attention（多头注意力）是Transformer架构的核心组件，也是GPT自注意力机制的关键改进。它解决了单头Scaled Dot-Product Attention只能捕捉单一维度语义关联的局限，通过将注意力拆分为多个“头”，让模型同时关注不同位置、不同类型的语义信息，大幅提升模型的表达能力和泛化能力。

其核心逻辑是：将输入的Q、K、V通过线性投影后拆分为多个头，每个头独立计算Scaled Dot-Product Attention，最后将所有头的输出拼接并通过线性投影融合，得到最终的注意力输出，全程保证输入与输出维度一致，便于模块串联。

二、Multi-Head Attention 核心原理

1. 核心思想

单头注意力仅能学习一种注意力分布，无法兼顾多维度语义（如语法结构、语义关联、指代关系等）。而多头注意力通过“拆分-独立计算-融合”的流程，让每个头专注于不同的语义维度，最终融合所有头的特征，实现更全面的注意力捕捉。

关键前提：输入特征维度d_model必须能被头数num_heads整除，即d_model = num_heads × d_k（d_k为每个头的特征维度），确保拆分后每个头的维度一致，便于并行计算和后续拼接。

2. 完整计算流程

以GPT自注意力（Q=K=V）为例，Multi-Head Attention的计算步骤如下：

1. 线性投影：将输入Q、K、V分别通过3个独立的线性层，映射到相同的d_model维度（保持输入输出维度一致）；
2. 拆分多头：将投影后的Q、K、V拆分为num_heads个头，维度从[batch_size, seq_len, d_model]转换为[batch_size, num_heads, seq_len, d_k]；
3. 独立计算注意力：每个头分别执行Scaled Dot-Product Attention，可结合因果掩码、填充掩码（复用Day9内容）；
4. 拼接多头：将所有头的注意力输出拼接，维度从[batch_size, num_heads, seq_len, d_k]还原为[batch_size, seq_len, d_model]；
5. 输出投影：将拼接后的特征通过线性层进行融合，得到最终的注意力输出，并加入Dropout防止过拟合。

三、Multi-Head Attention 实现代码

1. 依赖函数（复用Day8、Day9实现）

依赖的Scaled Dot-Product Attention和掩码函数

```python
import torch
import torch.nn as nn
import math

# 复用Day8：Scaled Dot-Product Attention
class ScaledDotProductAttention(nn.Module):
    def __init__(self, dropout=0.1):
        super().__init__()
        self.dropout = nn.Dropout(dropout)

    def forward(self, q, k, v, mask=None):
        d_k = q.size(-1)
        scores = torch.matmul(q, k.transpose(-2, -1)) / math.sqrt(d_k)

        # 应用掩码（因果掩码/填充掩码）
        if mask is not None:
            scores = scores.masked_fill(mask, -1e9)  # 用-1e9避免数值问题

        attn_weights = self.dropout(torch.softmax(scores, dim=-1))
        output = torch.matmul(attn_weights, v)

        return output, attn_weights

# 复用Day9：因果掩码和填充掩码创建函数
def create_causal_mask(seq_len, device=None):
    # 因果掩码：屏蔽未来位置，形状[1, seq_len, seq_len]
    mask = torch.triu(torch.ones(seq_len, seq_len, device=device), diagonal=1).bool()
    return mask.unsqueeze(0)

def create_attn_padding_mask(seq, pad_idx):
    # 适用于注意力分数矩阵的填充掩码，形状[batch_size, seq_len, seq_len]
    batch_size, seq_len = seq.shape
    mask = (seq == pad_idx).unsqueeze(1)
    return mask.expand(batch_size, seq_len, seq_len)
```

2. Multi-Head Attention 核心类实现

Multi-Head Attention 完整实现

```python
class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, num_heads, dropout=0.1):
```

```python
        """
        初始化多头注意力层
        Args:
            d_model: 输入/输出特征维度（必须能被num_heads整除）
            num_heads: 注意力头的数量
            dropout: Dropout概率，用于防止过拟合
        """
        super().__init__()
        # 校验d_model是否能被num_heads整除，确保每个头维度一致
        assert d_model % num_heads == 0, "d_model must be divisible by num_heads"

        self.d_model = d_model          # 总特征维度
        self.num_heads = num_heads      # 注意力头数
        self.d_k = d_model // num_heads # 每个头的特征维度

        # 定义线性层：Q、K、V投影层 + 输出融合层
        self.w_q = nn.Linear(d_model, d_model)  # Q投影
        self.w_k = nn.Linear(d_model, d_model)  # K投影
        self.w_v = nn.Linear(d_model, d_model)  # V投影
        self.w_o = nn.Linear(d_model, d_model)  # 输出融合投影

        # 注意力计算依赖（复用ScaledDotProductAttention）
        self.attention = ScaledDotProductAttention(dropout=dropout)
        self.dropout = nn.Dropout(dropout)      # 输出Dropout

    def split_heads(self, x, batch_size):
        """
        将输入拆分为多个头，维度转换：[batch_size, seq_len, d_model] → [batch_size, num_heads, seq_len, d_k]
        Args:
            x: 输入张量，形状[batch_size, seq_len, d_model]
            batch_size: 批次大小
        Returns:
            拆分后的多头张量
        """
        # 先拆分d_model为num_heads × d_k，再转置调整维度顺序（便于多头并行计算）
        x = x.view(batch_size, -1, self.num_heads, self.d_k)
        return x.transpose(1, 2)  # 转置后：[batch_size, num_heads, seq_len, d_k]

    def combine_heads(self, x, batch_size):
        """
        将多个头的输出拼接融合，维度转换：[batch_size, num_heads, seq_len, d_k] → [batch_size, seq_len, d_model]
        是split_heads的逆操作
        Args:
```

```
            x: 多头注意力输出，形状[batch_size, num_heads, seq_len, d_k]
            batch_size: 批次大小
        Returns:
            拼接融合后的张量
        """
        # 先转置还原维度顺序，再拼接为d_model维度
        x = x.transpose(1, 2)  # 转置后：[batch_size, seq_len, num_heads, d_k]
        # contiguous()确保内存连续，避免view报错
        return x.contiguous().view(batch_size, -1, self.d_model)

    def forward(self, q, k, v, mask=None):
        """
        前向传播（自注意力模式：Q=K=V）
        Args:
            q: 查询张量，形状[batch_size, seq_len_q, d_model]
            k: 键张量，形状[batch_size, seq_len_k, d_model]
            v: 值张量，形状[batch_size, seq_len_v, d_model]（seq_len_k = seq_len_v）
            mask: 掩码张量，形状[batch_size, seq_len_q, seq_len_k]，可选（因果/填充掩码）
        Returns:
            output: 多头注意力最终输出，形状[batch_size, seq_len_q, d_model]
            attn_weights: 所有头的注意力权重，形状[batch_size, num_heads, seq_len_q, seq_len_k]
        """
        batch_size = q.size(0)

        # 步骤1：Q、K、V线性投影
        q_proj = self.w_q(q)  # [batch_size, seq_len_q, d_model]
        k_proj = self.w_k(k)  # [batch_size, seq_len_k, d_model]
        v_proj = self.w_v(v)  # [batch_size, seq_len_v, d_model]

        # 步骤2：拆分多头
        q_split = self.split_heads(q_proj, batch_size)  # [batch_size, num_heads, seq_len_q, d_k]
        k_split = self.split_heads(k_proj, batch_size)  # [batch_size, num_heads, seq_len_k, d_k]
        v_split = self.split_heads(v_proj, batch_size)  # [batch_size, num_heads, seq_len_v, d_k]
```

3. 测试代码（验证功能正确性）

Multi-Head Attention 测试

```
if __name__ == "__main__":
    # 设备配置：自动选择GPU/CPU
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    print(f"使用设备：{device}")
```

```python

    # 超参数设置（GPT基础配置）
    batch_size = 2     # 批次大小
    seq_len = 5        # 序列长度
    d_model = 512      # 总特征维度
    num_heads = 8      # 注意力头数（512 ÷ 8 = 64，每个头64维）
    pad_idx = 0        # PAD token索引

    # 模拟输入：文本序列嵌入后的张量（实际由Embedding层生成）
    q = torch.randn(batch_size, seq_len, d_model).to(device)
    k = torch.randn(batch_size, seq_len, d_model).to(device)
    v = torch.randn(batch_size, seq_len, d_model).to(device)

    # 模拟变长序列（含PAD token），用于生成填充掩码
    seq = torch.tensor([[1, 2, 3, 0, 0], [4, 5, 0, 0, 0]]).to(device)

    # 步骤1：创建掩码（结合因果掩码和填充掩码，适配GPT自注意力）
    causal_mask = create_causal_mask(seq_len, device)  # [1, 5, 5]
    padding_mask = create_attn_padding_mask(seq, pad_idx)  # [2, 5, 5]
    combined_mask = torch.logical_or(causal_mask, padding_mask)  # 组合掩码

    # 步骤2：初始化多头注意力层
    mha = MultiHeadAttention(d_model=d_model, num_heads=num_heads).to(device)

    # 步骤3：前向传播（自注意力模式，Q=K=V）
    output, attn_weights = mha(q, k, v, mask=combined_mask)

    # 步骤4：验证输出形状和掩码效果
    print(f"\n输入形状：{q.shape}")                    # [2, 5, 512]
    print(f"输出形状：{output.shape}")               # [2, 5, 512]（维度不变，可串联）
    print(f"注意力权重形状：{attn_weights.shape}")  # [2, 8, 5, 5]（batch, heads, seq_q, seq_k）

    # 验证掩码效果：PAD位置和未来位置的注意力权重接近0
    print("\n第一个样本、第一个头的注意力权重（保留2位小数）:")
    print(attn_weights[0, 0].round(2))
    print("\n第一个样本注意力权重每行求和（应为1，验证softmax有效性）:")
    print(attn_weights[0].sum(dim=-1).round(2))
```

四、关键细节解析

1. 拆分与拼接逻辑

split_heads和combine_heads是多头注意力的核心操作，二者互为逆操作，核心目的是让每个头独立计算注意力，同时保证输入输出维度一致：

- split_heads：通过view拆分维度、transpose调整顺序，将“批次-序列-特征”转换为“批次-头-序列-特征”，让每个头的特征独立，便于并行计算；
- combine_heads：先transpose还原顺序，再用view拼接特征，contiguous()确保张量内存连续，避免view操作报错（PyTorch中view要求张量内存连续）。

2. 掩码适配技巧

由于多头注意力的输入维度多了“头”这一维度，掩码需要适配才能正常广播：

原始掩码形状（如因果掩码[1, seq_len, seq_len]、填充掩码[batch_size, seq_len, seq_len]）通过unsqueeze(1)增加一个维度，变为[1, 1, seq_len, seq_len]或[batch_size, 1, seq_len, seq_len]，可自动广播到所有num_heads个头上，确保每个头都应用相同的掩码规则。

3. 线性投影的作用

多头注意力中的4个线性层（w_q、w_k、w_v、w_o）各司其职，不可省略：

- w_q、w_k、w_v：将输入特征投影到相同的d_model维度，为拆分多头做准备，同时学习Q、K、V的特征映射；
- w_o：将多个头的拼接结果进行融合，学习不同头特征的最优组合，而非简单拼接，提升模型表达能力。

五、关键注意事项

1. **维度校验**：必须确保d_model能被num_heads整除，否则无法均匀拆分多头，会导致代码报错；常用配置（d_model=512, num_heads=8；d_model=1024, num_heads=16）。
2. **掩码广播**：掩码的形状必须适配注意力分数矩阵和多头维度，否则会出现广播失败的问题，核心是保证掩码的“头”维度为1，便于广播。
3. **数值稳定性**：注意力分数掩码填充时，优先使用-1e9而非float('-inf')，避免某些计算场景下出现NaN或数值溢出问题。
4. **与后续模块的衔接**：Multi-Head Attention的输出维度与输入一致（[batch_size, seq_len, d_model]），可直接串联Day11要学习的LayerNorm、残差连接和FFN层。
5. **头数选择**：头数并非越多越好，头数过多会增加计算量和过拟合风险，GPT-2使用12头，GPT-3使用96头，需根据模型规模和任务调整。

六、与前后学习内容的关联

1. 依赖基础：Day8的Scaled Dot-Product Attention是多头注意力的核心单元，每个头的注意力计算都依赖该模块；Day9的掩码机制（因果掩码、填充掩码）用于屏蔽无效信息，确保多头注意力的合理性。

2. 后续衔接：Day11将学习LayerNorm、Dropout和残差连接，与本文实现的Multi-Head Attention串联，构成完整的Transformer Block，为后续手写小GPT的解码器打下基础。

（注：文档部分内容可能由 AI 生成）